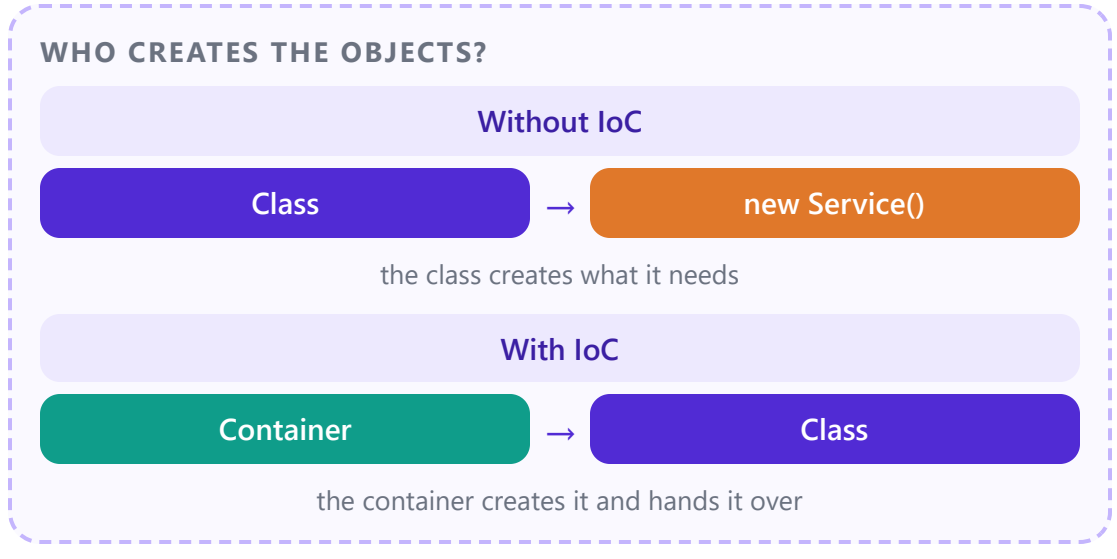
IoC Container

The container behind every `builder.Services.AddXxx()`

1 What is IoC?

IoC stands for **Inversion of Control**. It means the control is **flipped**.

- **Without IoC:** the class is in control. It creates what it needs (`new`).
- **With IoC:** another party is in control. It creates the objects and hands them to the class.



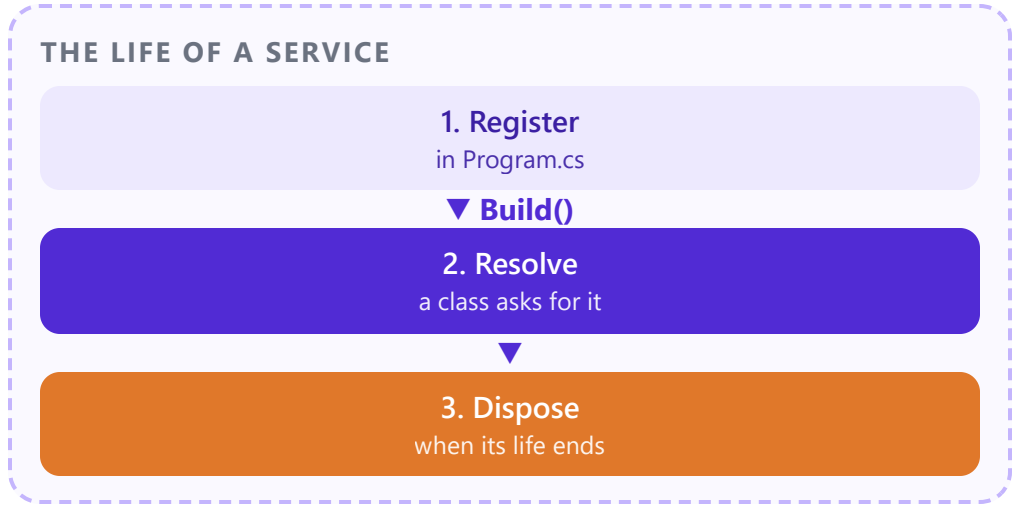
2 What is the IoC Container?

That other party. A **container** that knows how to create objects, keeps them, and hands them to whoever asks.

What does the container do?

Three jobs

Job	What it means	Where we see it
Register	We tell it what to create, and for which type	`builder.Services.AddXxx()`
Resolve	It creates the object and hands it over on request, building the whole chain it needs	Automatically, in the constructor
Dispose	It disposes of the objects it created when their life ends	Automatically



> **Who creates it?** `WebApplication.CreateBuilder` prepares the container, and we register in it through `builder.Services`. At `Build()` the container is built, and we **cannot register after it**.

Ways to register

Four forms

1. A TYPE AND ITS IMPLEMENTATION

```
builder.Services.AddScoped<IPaymentMethod, CashPayment>();
```

"When `IPaymentMethod` is requested, give `CashPayment`". The most common form.

2. A CLASS WITHOUT AN INTERFACE

```
builder.Services.AddScoped<Connection>();
```

The class itself is requested and created.

3. WITH A FACTORY

```
builder.Services.AddScoped<IPaymentMethod>(
    sp => new CardPayment());
```

When creating the object needs special logic. `sp` is the `IServiceProvider`, used to get other services if needed.

4. A READY OBJECT (SINGLETON ONLY)

```
builder.Services.AddSingleton<IClock>(
    new SystemClock());
```

We create the object ourselves and hand it to the container to keep. `IClock` and `SystemClock` are your own classes.

Lifetimes

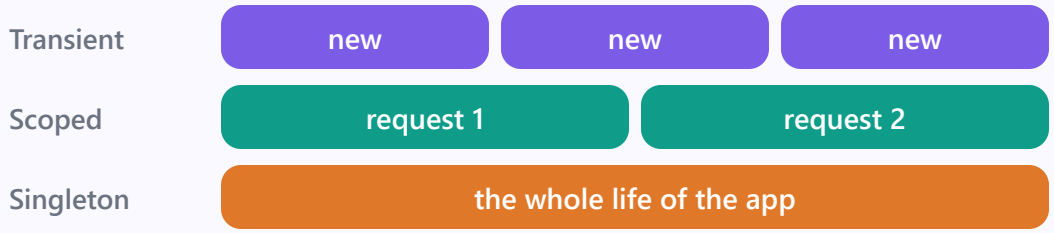
How long does an object live?

Method	Object lifetime
`AddTransient`	A **new** object every time it is requested
`AddScoped`	One object per **HTTP request**
`AddSingleton`	One object for the **whole life** of the app

WHEN TO USE WHICH

- **Transient:** a light service with no state.
- **Scoped:** a service tied to one request, like database access.
- **Singleton:** something expensive to create or shared by everyone, like a cache. It must be **thread-safe**.

LIFETIME ON A TIMELINE



AN ANALOGY: A RESTAURANT

Transient
a new paper cup every time you order a drink

Scoped
one cup per customer for the whole visit; the next customer gets a new one

Singleton
one coffee machine for the whole restaurant

The example: one class, one interface

We change only the registration line, and test each lifetime in turn

1 The interface and the class

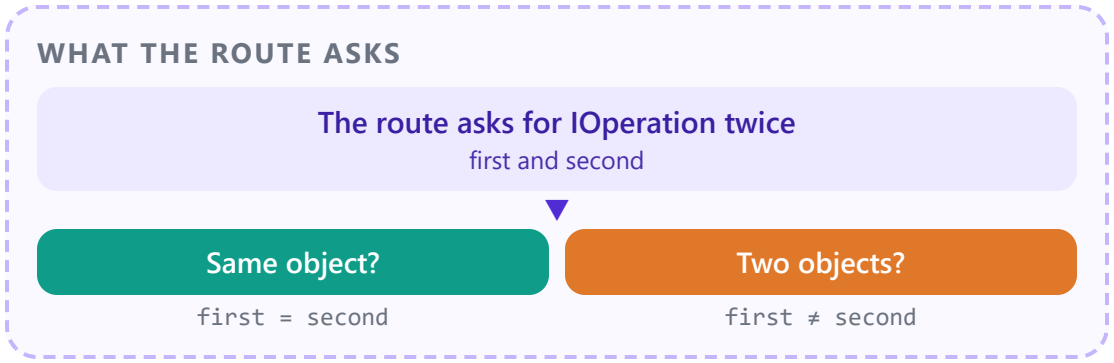
The class makes a `Guid` **once, when the object is created**. The same number twice means the **same object**. A different number means a **new object**.

```
public interface IOperation
{
    Guid Id { get; }
}

public class Operation : IOperation
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

2 The route, after Build()

```
app.MapGet("/operation", (IOperation first, IOperation second) =>
    $"first = {first.Id.ToString()[..8]}   second =
{second.Id.ToString()[..8]}");
```



3 How to run each experiment

- Change **only** the registration line in `Program.cs`.
- Run the app.
- Open `/operation` **twice** (refresh the page).
- Compare `first` and `second`, then compare request 1 with request 2.

The three experiments

Same class, same route. Only the registration changes.

EXPERIMENT 1: TRANSIENT

```
builder.Services.AddTransient<IOperation, Operation>();
```

```
Request 1: first = b5edf808   second = 189f0bac
Request 2: first = 3410371c   second = 93d5c59f
```

first and second are different inside one request: a new object every time it is asked. The numbers also change in request 2.

EXPERIMENT 2: SCOPED

```
builder.Services.AddScoped<IOperation, Operation>();
```

```
Request 1: first = 390c163f   second = 390c163f
Request 2: first = 6bde7177   second = 6bde7177
```

first and second are the same inside one request: one object shared. The number **changes** in request 2: a new object per request.

EXPERIMENT 3: SINGLETON

```
builder.Services.AddSingleton<IOperation, Operation>();
```

```
Request 1: first = 5f708413   second = 5f708413
Request 2: first = 5f708413   second = 5f708413
```

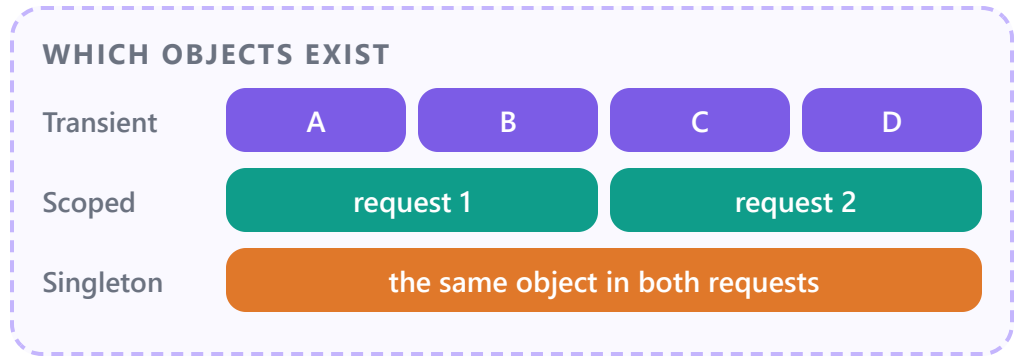
The same number everywhere, in both requests: one object for the whole life of the app. Restart the app and a new number appears.

Comparing the three lifetimes

What the experiments showed

Lifetime	Inside one request (first and second)	Between two requests
Transient	**Different**	Changes
Scoped	**Same**	**Changes**
Singleton	**Same**	**Never changes**

- **Transient:** every ask is a new object, even inside one request.
- **Scoped:** one object shared inside a request, renewed in the next.
- **Singleton:** one object for the whole app.



Each box is **one object**. Transient made four objects for two requests (two per request). Scoped made two. Singleton made one.

Captive Dependency

A mistake with lifetimes

What is it?

A **longer-living** object depends on a **shorter-living** one. For example, a Singleton asks for a Scoped service in its constructor. The Singleton lives for the whole app, but it keeps a Scoped object that should have died with its request. It **captures** it.

```
builder.Services.AddScoped<ScopedService>();
builder.Services.AddSingleton<SingletonService>();

public class SingletonService
{
    public SingletonService(ScopedService scoped) { }
}
```

IN DEVELOPMENT THE APP STOPS AT STARTUP

```
Cannot consume scoped service 'ScopedService'
from singleton 'SingletonService'.
```

> **Rule:** an object depends only on objects that live **as long as it does, or longer**.

Depends on →	Transient	Scoped	Singleton
Transient	Allowed	Allowed	Allowed
Scoped	Allowed	Allowed	Allowed
Singleton	Allowed*	**Error**	Allowed

* it stays alive as long as the Singleton

More than one implementation

Several classes registered for the same interface

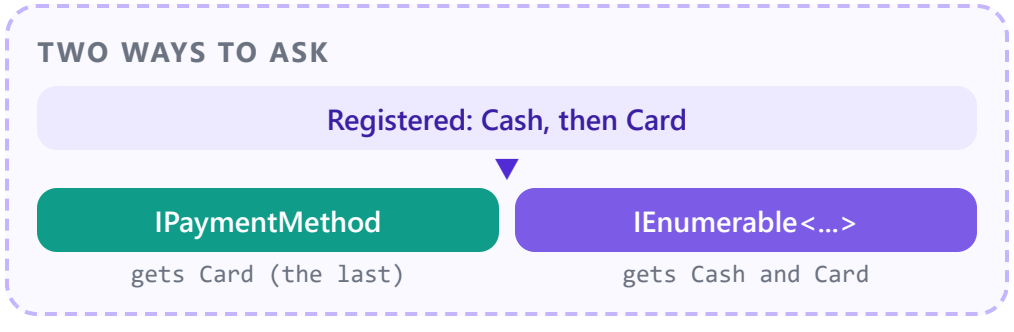
```
builder.Services.AddScoped<IPaymentMethod, CashPayment>();
builder.Services.AddScoped<IPaymentMethod, CardPayment>();

app.MapGet("/many", (IPaymentMethod single,
                     IEnumerable<IPaymentMethod> all) => new
{
    Single = single.Pay(),
    All = all.Select(p => p.Pay())
});
```

RESULT

```
{"single":"Paid with card",
 "all":["Paid with cash","Paid with card"]}
```

- Asking for `IPaymentMethod` alone gives the **last registration** (`CardPayment`).
- Asking for `IEnumerable<IPaymentMethod>` gives **all** implementations, in registration order.



TryAdd

Registers only **if the type is not registered yet**. It does nothing otherwise, so the existing registration stays. The result above stayed `CardPayment`.

```
using Microsoft.Extensions.DependencyInjection.Extensions;

builder.Services.TryAddScoped<IPaymentMethod, CashPayment>();
```

Keyed services and manual resolving

Pick one implementation, or ask the container yourself

Keyed services

When we want **one specific** implementation, we register each with a key:

```
builder.Services.AddKeyedScoped<IPaymentMethod, CashPayment>
("cash");
builder.Services.AddKeyedScoped<IPaymentMethod, CardPayment>
("card");

using Microsoft.AspNetCore.Mvc;

app.MapGet("/keyed",
    ([FromKeyedServices("card")] IPaymentMethod payment) =>
payment.Pay());
```

Result: `Paid with card`.

Getting a service by hand

The normal way is the constructor. Sometimes we need the container itself, through `IServiceProvider`:

```
app.MapGet("/manual", (IServiceProvider sp) =>
{
    using IServiceScope scope = sp.CreateScope();
    IPaymentMethod payment =
        scope.ServiceProvider.GetRequiredService<IPaymentMethod>();
    return payment.Pay();
});
```

- `CreateScope()` creates a new scope. Scoped objects are created inside it and disposed at the end of the `using`.
- `GetRequiredService` throws if there is no registration. `GetService` returns `null`.

> **Service Locator.** Asking `IServiceProvider` for services inside your classes, instead of the constructor, hides what the class needs and makes it harder to test. Use it only when necessary.

Disposing and organizing

Who cleans up, and how to keep Program.cs short

Dispose

The container **disposes** of every `IDisposable` object **it created**, when its life ends.

```
public class Connection : IDisposable
{
    public Connection() => Console.WriteLine("Connection
created");
    public void Dispose() => Console.WriteLine("Connection
disposed");
}

builder.Services.AddScoped<Connection>();
app.MapGet("/dispose", (Connection connection) => "done");
```

CONSOLE OUTPUT

```
Connection created
Connection disposed
```

- A **Scoped** object is disposed at the **end of the request**.
- A **Singleton** is disposed when the **app stops**.
- Do not call `Dispose` on an object the container created. An object you created with `new` and handed over is **not** disposed by the container.

Organizing registrations

With many services, `Program.cs` gets crowded. We gather them in an **extension method**:

```
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddPaymentServices(
        this IServiceCollection services)
    {
        services.AddScoped<IPaymentMethod, CashPayment>();
        services.AddScoped<Customer>();
        return services;
    }
}
```

IN PROGRAM.CS

```
builder.Services.AddPaymentServices();
```

Summary

What to remember, and what to try

SUMMARY

Concept	What it is
IoC	Inversion of control: another party creates the objects
IoC Container	Registers, resolves and disposes
Registration	A type and its implementation, a class, a factory, or a ready object
Lifetimes	Transient: always new. Scoped: per request. Singleton: the whole app.
Captive Dependency	A Singleton depending on a Scoped service is an error
Many implementations	The last registration wins. `IEnumerable` gives all.
Keyed services	Register with a key, and ask for a specific one
Dispose	The container disposes of what it created

TRY IT YOURSELF

- Run the three lifetime experiments. In each one open `/operation` twice and write down what is the same and what is different.
- Make `SingletonService` ask for `ScopedService` in its constructor, and write down the error.
- Register two implementations of `IPaymentMethod` and make a route that returns the results of `Pay()` for all of them through `IEnumerable`.
- Try Keyed services to choose `cash` or `card` from the URL.

REMEMBER THE PICTURE

